# The Modify Right as Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern

**Author:** Wenxin Li (Independent Researcher)
**ORCID:** 0009-0004-8065-3235
**Date:** 1 May 2026


---

## Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the three rights named in the source paper's "human-governed" commitment — the **modify right** — as a standalone architectural commitment with independent operational content, separable from the inspect and override rights with which it composes.

## Abstract

The Coordination Knowledge Substrate (CKS) pattern's "human-governed" commitment names three rights — to inspect, to modify, and to override substrate content and orchestration rules — as jointly necessary for the architectural property the term carries. A separate note formalizes that joint commitment, and a sibling note formalizes the inspect right as standalone. This note formalizes the modify right as having independent operational content that can be defended, implemented, and tested independently of the inspect and override rights. Two motivations carry the standalone treatment. First, modify-only roles such as content curators, domain-expert editors, data-entry operators, substrate authors, and scribes exercise primarily the modify right without exercising override authority, and the architecture must describe their roles as CKS-coherent governance. Second, the modify right is operationally distinct from the override right: modification within the authority structure the deployment configures is the routine operation that produces ongoing substrate growth, while override is the architectural escape hatch. The note states what the modify right requires, distinguishes it from three adjacent commitments commonly conflated with it (database write access, content-management edit permissions, agent-framework write authority), enumerates the failure modes that violate it, and provides an operational test for whether a system implements the modify right separable from the broader human-governed commitment.

## 1. Why the modify right needs to be formalized as standalone

The CKS pattern's "human-governed" commitment names three rights together: to inspect, to modify, and to override substrate content and orchestration rules at any time during the substrate's existence (§2.1, §3.3 of the source paper). A separate derivation note formalizes that joint commitment as an authority architecture rather than a labor or review commitment, and a sibling note formalizes the inspect right as having independent operational content within the joint commitment.

Two motivations carry the standalone treatment of the modify right.

The first is that the architecture must describe modify-only governance roles. In ongoing substrate operations, many humans exercise authority primarily as writers without exercising override authority over existing decisions. Content curators populate substrate content within an authorized scope; domain-expert editors refine descriptions, correct errors, or extend rationale fields; data-entry operators record new entities and relationships as they arise; substrate authors draft orchestration rules within their authority; scribes record decisions reached in meetings that occurred outside the substrate. None of these roles is exercising override authority — they are not bypassing orchestration rules or contradicting prior decisions; they are doing the routine work of populating, updating, and maintaining substrate content. If the human-governed commitment is treated as a single composite, these roles look like "limited override" or "constrained authority," which mischaracterizes what they actually do. The architecture needs language for the routine writer that does not borrow vocabulary from the escape-hatch case.

The second motivation is the modify-versus-override distinction itself. Most substrate writes are not overrides. A modifier who edits substrate content within their authorized scope is exercising routine write authority within the authority structure the deployment's governance configuration permits; override means making changes that bypass orchestration rules, contradict default processes, or supersede LLM-produced output that would otherwise stand. The two operations sit at different points in the architecture's cost profile. Modification is the routine operation; override is the exception. Conflating them inflates governance cost — every substrate write becomes architecturally exceptional, which the linear-cost commitment cannot afford (§6.3) — and dilutes the override right's distinctive meaning as the architectural escape hatch. Naming the modify right as standalone is what preserves both framings.

## 2. The modify right, defined precisely

In the CKS pattern, a human exercises the **modify right** when they create, change, or delete substrate content or orchestration-rule content directly, without LLM intermediation as a precondition, within the authority scope the deployment grants them, with the change taking effect as substrate state. The right has five operational components.

**(a) Write access to substrate content within authorized scope.** A human with appropriate access can create new substrate content, modify existing content, and delete content where authorized — entities, relationships, decisions, rationale, conflicts, provenance fields. Authorization scope is determined by the broader human-governed authority architecture; the modify right operates within whatever scope is authorized.

**(b) Write access to orchestration rules within authorized scope.** A human with appropriate access can create new orchestration rules, modify existing rules, and deprecate rules within authorized scope. Authority to author or modify rules is itself governance configuration; not every modifier holds rule-write authority. The right's content does not require that every modifier hold every form of write authority — only that within the partition where the right applies, the components named here all hold.

**(c) Direct access without LLM intermediation as a precondition.** The human may use LLM-assisted tools to draft, edit, format, or stage changes, and such tools are often valuable. But the human must also be able to write directly when they choose, with their own attribution, without the LLM as a gate. The LLM is permissible as an adjacent tool; it cannot be the only path through which the human's writes become substrate state.

**(d) Effective writes — changes take effect as substrate state.** A modification within authorized scope takes effect as substrate state directly, with appropriate provenance recorded, on the modifier's authority alone. A "modification" that produces only a pending change requiring further automated approval before becoming state, or that is recorded somewhere but does not flow into the substrate's authoritative content, does not exercise the modify right at architectural scope. Audit logs, peer review, and other adjacent processes may surround modifications without becoming preconditions of effect; what the right requires is that within the partition where it applies, writes become state directly.

**(e) Within authorized scope.** The right operates within the authority structure the deployment configures. Modification authority can be partitioned by substrate region, content type, rule category, role, or other dimensions; the right is not a commitment to unrestricted write capability across the substrate. What the right requires is that within the partition where it applies, the components above hold.

The five components together define what the modify right requires of the host environment and the substrate's representational form. Failing any one — even with the other four robustly satisfied — fails the modify right architecturally.

## 3. What the modify right does NOT require

The standalone treatment of the modify right is not a maximalist treatment. Stating precisely what the right does not require is what keeps the standalone framing from drifting beyond what the source paper supports.

**It does not require inspect access beyond the modifier's working scope.** A modifier who writes substrate content in a specific area does not architecturally need read access to all substrate content; the deployment may grant the inspect right at a different scope. The two rights are separate partitions, and a deployment may configure them independently.

**It does not require override authority.** A modifier who writes substrate content within authority scope but cannot bypass orchestration rules or contradict prior decisions outside that scope is still exercising the modify right at full architectural content. Override is the third of the three rights, separable from modify in the same sense modify is separable from inspect.

**It does not require justification.** Modifications within authority scope take effect on the modifier's authority alone; they are not architecturally gated on rationale, approval, or comment. Audit logging, peer review, and other adjacent processes may record context around modifications without being preconditions of effect. (The no-justification property is shared with the override right and is what distinguishes architectural authority from process authority; a system that requires justification before writes take effect has substituted process authority for architectural authority.)

**It does not require comprehension of consequence.** The modify right is the act of writing within authority, not the wisdom of the writing. Whether a modification is well-considered, beneficial, or correctly reasoned is outside the architecture's scope. Such considerations may be addressed by adjacent processes — review, training, escalation — but they are not preconditions of the right's exercise.

**It does not require continuous engagement.** The right is exercisable at any time the modifier chooses; it imposes no obligation to maintain substrate content on any schedule, in response to any event, or at any frequency. A modifier who exercises the right once and never again has exercised it as fully, architecturally, as one who writes continuously.

## 4. What the modify right is NOT

Three adjacent commitments are commonly conflated with the modify right. Each is a real and reasonable commitment in some other architecture; naming what the modify right is not is what prevents the misreading.

**Not database write access.** Database write access is a host-level capability — the user's account holds permission to issue INSERT, UPDATE, or DELETE statements against tables, with the operation succeeding subject to the database engine's checks. The modify right is architectural rather than host-level. It commits to direct human authority over substrate content as a property of the architecture, with provenance recorded, scoped within the governance configuration, exercisable without scheduling. A deployment can have full database write access and still fail the modify right — for instance, if the database is gated behind an LLM that interposes itself, or behind an approval queue or workflow whose intermediate stages can suppress the writes. Database write access is a permissible host-level mechanism for the modify right, but the right's architectural content is not satisfied by host-level access alone.

**Not edit permissions in content-management systems.** Edit permissions in CMSes are typically tied to roles and workflows — a contributor edits a draft, an editor approves it, a publisher pushes it live. The modify right does not commit to or against workflow architectures; what it commits to is that the human writing substrate content writes substrate state directly, with the change effective. CMS-style multi-stage review can be layered on top of a substrate that satisfies the modify right when the underlying write capability is direct; it cannot itself constitute the modify right when the underlying writes are gated as preconditions of effect rather than recorded as adjacent context around effective writes.

**Not write authority in agent frameworks.** Some agent frameworks allow agents to "modify" memory or knowledge stores under human-configured policies. This is structurally different from the modify right. Agent-framework write authority is a policy granted to a non-human actor; the modify right is human authority over substrate content, with the human's identity recorded as writer in the substrate's provenance. The architectural commitment is to human write capability specifically, not to write capability in general. The distinction connects to the source paper's AI-as-substrate-mediator commitment (§4.1, §4.2): the LLM operates over the substrate as mediator under human-authored orchestration rules, not as an autonomous writer with its own authority. LLM-mediated writes recorded under appropriate provenance fall under the labor-allocation framework's mode 2 (LLM under human

direction); the modify right is what mode 1 (direct human writing) and mode 3 (stable cells under orchestration rules authored by humans) presuppose at the architectural layer.

The three distinctions together are what keep the modify right's content precise: the right is direct human authority over substrate content within scope, taking effect as substrate state, not host-level write capability, workflow-stage progression, or agent-policy authority.

## 5. What the modify right makes possible at the architectural layer

Naming the modify right as standalone architectural commitment has four consequences at the architectural layer.

**Modify-only governance roles become describable.** Content curators, domain-expert editors, data-entry operators, substrate authors, scribes, and other writer roles can be granted modify access within authorized scope, without override authority over existing decisions, and the architecture treats their exercise of authority as CKS-coherent governance. This gives downstream implementations a principled vocabulary for separation-of-duties configurations in which the routine writer is distinct from the override-bearing role.

**Non-specialist modification follows from tool-agnosticism.** The source paper's tool-agnosticism commitment (§7.1) names three minimal requirements for any environment that instantiates the pattern: persistent structured state, human read/write access, and LLM access to substrate content. The second of these grounds the modify right's direct-access component. Because the right is satisfied in any environment meeting that requirement, it is exercisable in commodity tools by humans without specialist training. A non-specialist editor updating a spreadsheet substrate is exercising the modify right at full architectural scope (§7.4).

**Routine substrate growth becomes architecturally tractable.** The substrate's accumulation of content over time depends on modify writes happening as the routine operation, not as escape-hatch overrides. Without the modify right as standalone, every substrate write would be architecturally exceptional, which the linear-cost commitment (§6.3) cannot afford — governance cost would scale with write frequency rather than with rule variety and intervention frequency. Naming the modify right as the routine operation is what keeps the cost profile coherent: routine writes are governance-cheap because they are exercises of architecturally normal authority, not exceptional interventions requiring per-write architectural expense.

**Human and LLM-mediated writes coexist within the labor-allocation framework.** The labor-allocation framework formalized as a sibling derivation note describes three modes by which substrate content is written: humans writing directly (mode 1), LLMs writing under human direction (mode 2), and stable cells largely automating writes under orchestration rules authored by humans (mode 3). All three modes write substrate content as routine operation, not as override; all three record provenance distinguishing the writer; all three operate within authority scope. The framework presupposes the modify right's standalone treatment, because without it the three modes would all be reframed as varieties of override — which inverts the cost profile and dissolves the architecture's distinction between routine operation and escape hatch.

These four are not new commitments; they follow from treating the right as having independent operational content.

## 6. Failure modes that violate the modify right

A system can fail the modify right specifically, even when it satisfies the inspect and override rights and broader governance commitments. Six failure modes name the most common ways this happens.

**(a) LLM-mediated modification.** When the only path to writing substrate content runs through an LLM call — the human "asks" the system to update the substrate, and the LLM produces the write — the right is violated. The LLM may be useful as an adjacent drafting tool; it cannot be the gate through which the human's authority becomes substrate state.

**(b) Approval-gated modification.** When writes within authorized scope require automated or process approval before taking effect as substrate state — that is, the "write" produces a pending change rather than effective state — the effective-writes component is violated. The right requires that writes within authority become state directly, not via further gating.

**(c) Scheduled-window modification.** When modification is permitted only during scheduled write windows, batch processing intervals, or approval cycles, the at-the-time-of-choosing component is violated. The right is exercisable when the modifier decides, not when a process allows.

**(d) Vendor-revocable modification.** When the host environment, vendor, or runtime middleware can in principle prevent a human from writing authorized substrate content, the right is architecturally compromised, regardless of how rarely the prevention occurs in practice. The architectural commitment is to a property of the system's design, not to a vendor's current policy. Modifiability cannot be revocable as a system feature.

**(e) Compiled or transformed write paths.** When substrate modifications must be expressed in a compiled, embedded, or otherwise transformed form before they can take effect — the modifier's input converted to embeddings as the underlying state, or to a vendor-specific binary that cannot be edited directly, or to a derived projection whose source is not human-writable — the direct-access component is violated. The substrate must be writable in inspectable form.

**(f) Write authority bound to LLM identity.** When the only writer the architecture recognizes is the LLM mediator — humans can ask the LLM to write but cannot write themselves with their own attribution — the right is violated. Human writes must be attributable to the human writer, not to the LLM acting on the human's behalf. This failure mode is structurally distinct from (a): (a) is about the LLM being the gate to writing; (f) is about the LLM being the only entity the architecture recognizes as writer, so even direct human writes lose their human attribution. The two can coincide but need not.

A system that exhibits any of (a)–(f) does not implement the modify right specifically, and naming the failure precisely is what allows downstream remediation.

## 7. Operational test

A system implements the modify right specifically if and only if all of the following are true at all times during the substrate's existence:

1. A human with appropriate access can write substrate content within authorized scope, in inspectable form, without LLM intermediation as a precondition.
2. Writes within authorized scope take effect as substrate state directly, with the human's identity recorded as writer in the substrate's provenance metadata.
3. Writes do not require scheduling, automated approval, or workflow gating as preconditions of effect; they are exercisable at the human's chosen time.
4. No LLM operation, vendor policy, or runtime middleware can in principle prevent (1), (2), or (3) for authorized humans.
5. Substrate content is writable in a form that becomes substrate state directly, not only through machine-mediated transformation of the writer's input.

A system that fails any of (1)–(5) does not implement the modify right specifically, even if it satisfies the inspect and override rights and the broader human-governed commitment in some other respect. Such a system is not CKS-coherent on the modification axis, and downstream work that relies on its modification guarantees should be scoped accordingly.

## 8. Conclusion

Implementations that conflate the modify right with the override right treat every substrate write as architecturally exceptional, producing systems where routine substrate growth is governance-expensive and where the override right loses its distinct meaning as the architectural escape hatch. Implementations that conflate the modify right with database write access miss the architectural content of the commitment — the directness, the within-authority-scope, the effective-writes property — and produce systems where human writes are technically possible but architecturally ungoverned. Implementations that fail the modify right specifically while claiming to satisfy human-governed produce systems where inspection and override are exercisable but routine modification is gated, which inverts the cost profile the architecture commits to.

Naming the modify right as standalone architectural commitment makes all three failure modes visible. It gives downstream implementers a precise specification of what their write mechanism must satisfy, independent of how inspection and override are handled. It makes modify-only writer roles describable as CKS-coherent governance rather than as partial or simulated authority. And it preserves the source paper's joint framing of human-governed: the standalone treatment names independent operational content within a composite right, not a fourth right alongside the three.

Subsequent work that implements, extends, or argues against the CKS modification commitment should use "the modify right" in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

---

**Source paper**

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

**How to cite this note**

Li, W. (2026). *The Modify Right as Standalone Architectural Commitment in the Coordination Knowledge Substrate Pattern.* 1 May 2026. ORCID: 0009-0004-8065-3235.